

Analysis of the Solution Approach for the Nonconvex Pricing Problem

1. General Approach

In this paper, the authors do not handle the nonconvexity by converting the problem into a convex problem. They also do not use local-optimization methods such as Successive Convex Approximation (SCA), Block Coordinate Descent (BCD), or other local solvers.

Instead, the authors exploit the structure of the Stackelberg game and the closed-form best response of the users. The solution approach is different for the uniform pricing case and the differentiated pricing case.

2. Uniform Pricing Case

In the uniform pricing case, the edge cloud charges one common price to all users:

$$\mu_1 = \mu_2 = \cdots = \mu_K = \mu.$$

Using backward induction, the authors first solve the users' best-response problem. For a fixed price μ , user k either offloads m_k bits if

$$\mu \leq \frac{1}{F_k},$$

or does not offload otherwise. This can be written as

$$x_k = \begin{cases} 1, & \mu \leq \frac{1}{F_k}, \\ 0, & \text{otherwise.} \end{cases}$$

Therefore, the optimal offloading decision of user k is

$$\ell_k^*(\mu) = m_k x_k.$$

The important point is that the user decision changes only at the threshold prices

$$\left\{ \frac{1}{F_k} \right\}_{k=1}^K.$$

Therefore, the continuous price search does not need to be solved using nonlinear programming. The authors reduce it into a finite one-dimensional search over these candidate prices.

The edge cloud revenue is

$$U_B(\mu) = \mu \sum_{k=1}^K m_k x_k C_k,$$

subject to the capacity constraint

$$\sum_{k=1}^K m_k x_k C_k \leq \bar{F}.$$

2.1. Uniform Pricing Algorithm

First, sort the users according to their threshold values:

$$\frac{1}{F_1} < \frac{1}{F_2} < \dots < \frac{1}{F_K}.$$

The algorithm checks the candidate prices from the largest threshold to the smallest:

$$\frac{1}{F_K}, \frac{1}{F_{K-1}}, \dots, \frac{1}{F_1}.$$

At price

$$\mu = \frac{1}{F_\tau},$$

the users who offload are users

$$\tau, \tau + 1, \dots, K.$$

Therefore, the total workload at candidate price $\mu = 1/F_\tau$ is

$$W_\tau = \sum_{k=\tau}^K m_k C_k.$$

The revenue at this candidate price is

$$U_\tau = \frac{1}{F_\tau} \sum_{k=\tau}^K m_k C_k.$$

The algorithm can be summarized as follows.

Initialization:

$$U_{\max} = 0,$$

$$\mu^* = 0,$$

$$x_k^* = 0, \quad \forall k.$$

Then, for

$$\tau = K, K-1, \dots, 1,$$

compute

$$W_\tau = \sum_{k=\tau}^K m_k C_k.$$

If

$$W_\tau \leq \bar{F},$$

then the candidate price is feasible. Compute

$$U_\tau = \frac{1}{F_\tau} W_\tau.$$

If

$$U_\tau > U_{\max},$$

then update

$$U_{\max} = U_\tau,$$

$$\mu^* = \frac{1}{F_\tau},$$

and

$$x_k^* = \begin{cases} 1, & k \geq \tau, \\ 0, & k < \tau. \end{cases}$$

If

$$W_\tau > \bar{F},$$

then the algorithm stops, because lowering the price further will include even more users and the workload will not decrease.

Therefore, for the uniform-pricing formulation, the algorithm gives the optimal uniform price. However, uniform pricing restricts the possible user sets. If a lower-threshold

user is selected, then all users with higher thresholds must also be selected. Because of this rule, a positive feasible offloading solution is not guaranteed. For example, if the highest-threshold user already has a workload larger than the cloud capacity, then the cloud cannot accept other lower-threshold users without also accepting that high-threshold user.

3. Differentiated Pricing Case

In the differentiated pricing case, the edge cloud can set a different price for each user:

$$\mu_k.$$

Here, once the cloud decides whether user k should be selected or not, the optimal price is already known. If user k is selected, then

$$x_k = 1.$$

The cloud sets the highest price that the user is still willing to accept:

$$\mu_k^* = \frac{1}{F_k}.$$

If user k is not selected, then

$$x_k = 0.$$

In that case, the cloud sets the price high enough so that the user does not offload.

This removes the continuous price variables and transforms the problem into a binary selection problem:

$$\max_{x_k \in \{0,1\}} \sum_{k=1}^K \frac{m_k x_k C_k}{F_k}$$

subject to

$$\sum_{k=1}^K m_k x_k C_k \leq \bar{F},$$

$$x_k \in \{0,1\}, \quad \forall k.$$

This is a 0-1 Integer Linear Programming (ILP) problem, specifically a binary knapsack problem.

4. Dynamic Programming Formulation

For the differentiated pricing problem, define the weight and value of user k as

$$w_k = m_k C_k,$$

and

$$v_k = \frac{m_k C_k}{F_k}.$$

Here,

$$w_k$$

is the cloud computation capacity consumed by user k , while

$$v_k$$

is the revenue obtained by selecting user k .

The differentiated pricing problem becomes

$$\max_{x_k \in \{0,1\}} \sum_{k=1}^K v_k x_k$$

subject to

$$\sum_{k=1}^K w_k x_k \leq \bar{F}.$$

This is exactly the structure of a 0-1 knapsack problem.

4.1. Integer or Discretized Capacity

Dynamic programming requires the capacity and weights to be represented as integer values. If w_k and $\bar{F}$ are already integers, then they can be used directly.

If they are continuous values, a discretization step is needed. For example, choose a discretization unit Δ , then define

$$\tilde{w}_k = \left\lceil \frac{w_k}{\Delta} \right\rceil,$$

and

$$\tilde{F} = \left\lceil \frac{\bar{F}}{\Delta} \right\rceil.$$

Then the dynamic programming table is built over the integer capacity index

$$f = 0, 1, 2, \dots, \tilde{F}.$$

For simplicity, the following formulation assumes the weights and capacity are already integer values. Therefore, w_k and $\bar{F}$ are used directly.

4.2. Dynamic Programming Table

Define the dynamic programming table as

$$DP[k][f],$$

where

$$k = 0, 1, 2, \dots, K,$$

and

$$f = 0, 1, 2, \dots, \bar{F}.$$

The meaning of $DP[k][f]$ is:

$$DP[k][f] = \text{maximum revenue using the first } k \text{ users with capacity limit } f.$$

4.3. Initialization

The initialization is

$$DP[0][f] = 0, \quad \forall f = 0, 1, \dots, \bar{F}.$$

This means that if no users are considered, the maximum revenue is zero.

Also,

$$DP[k][0] = 0, \quad \forall k = 0, 1, \dots, K.$$

This means that if the available capacity is zero, no user can be selected, so the revenue is zero.

Therefore, the initialization is

$$DP[0][f] = 0,$$

$$DP[k][0] = 0.$$

4.4. Recurrence Relation

For each user k , and for each capacity level f , there are two choices.

Choice 1: Do not select user k

If user k is not selected, then

$$x_k = 0.$$

The value remains

$$DP[k-1][f].$$

Choice 2: Select user k

If user k is selected, then

$$x_k = 1.$$

This is possible only if the weight of user k fits into the available capacity:

$$w_k \leq f.$$

If user k is selected, the remaining capacity is

$$f - w_k,$$

and the total value becomes

$$DP[k-1][f - w_k] + v_k.$$

Therefore, if

$$w_k \leq f,$$

the recurrence is

$$DP[k][f] = \max(DP[k-1][f], DP[k-1][f - w_k] + v_k).$$

If

$$w_k > f,$$

then user k cannot be selected, so

$$DP[k][f] = DP[k-1][f].$$

4.5. Final Optimal Value

After filling the table, the global optimal revenue is

$$DP[K][\bar{F}].$$

This value represents the maximum revenue that the edge cloud can obtain under the computation capacity constraint.

5. Recovering the Selected Users

To recover which users are selected, start from

$$k = K,$$

and

$$f = \bar{F}.$$

Then move backward through the table.

If

$$DP[k][f] = DP[k-1][f],$$

then user k was not selected, so

$$x_k^* = 0.$$

Then move to

$$k \leftarrow k - 1.$$

However, if

$$DP[k][f] \neq DP[k-1][f],$$

then user k was selected, so

$$x_k^* = 1.$$

Then update the remaining capacity:

$$f \leftarrow f - w_k,$$

and move to

$$k \leftarrow k - 1.$$

Repeat this process until

$$k = 0.$$

The result is the optimal selected user set:

$$x_1^*, x_2^*, \dots, x_K^*.$$

6. Recovering the Optimal Prices

After obtaining the optimal selection decision x_k^* , the optimal price for each user is

$$\mu_k^* = \begin{cases} \frac{1}{F_k}, & x_k^* = 1, \\ \infty, & x_k^* = 0. \end{cases}$$

The value

$$\mu_k^* = \infty$$

does not mean that the cloud literally charges an infinite price. It means that the cloud sets the price high enough so that user k will not offload.

Therefore, for selected users, the cloud charges the maximum acceptable price:

$$\mu_k^* = \frac{1}{F_k}.$$

For rejected users, the cloud sets a price above the user's threshold so that

$$\mu_k^* > \frac{1}{F_k}.$$

7. Complexity

The dynamic programming approach has time complexity

$$O(K\bar{F}),$$

assuming integer weights and integer capacity.

The space complexity is

$$O(K\bar{F})$$

if the full table is stored.

If only the optimal value is needed, the space complexity can be reduced to

$$O(\bar{F})$$

using a one-dimensional dynamic programming array. However, if the selected users must be recovered, then either the full table or an additional decision table is needed.

The dynamic programming method gives the global optimum for the reformulated differentiated-pricing problem. It is not a local solver like SCA or BCD. However, its complexity is pseudo-polynomial, because it depends on the numerical value of the capacity $\bar{F}$, not only on the number of users K .

8. Conclusion

The authors do not convexify the original nonconvex problem. Instead, they solve it by structural reformulation.

For the uniform-pricing case, they reduce the problem to a finite search over threshold prices:

$$\left\{ \frac{1}{F_k} \right\}_{k=1}^K.$$

For the differentiated-pricing case, they reformulate the problem as a 0-1 knapsack problem, which is a special type of Integer Linear Programming problem. The variables and parameters are:

$$x_k \in \{0, 1\},$$

$$w_k = m_k C_k,$$

$$v_k = \frac{m_k C_k}{F_k},$$

$$\bar{F} = \text{edge cloud computation capacity.}$$

The authors then solve the reformulated knapsack problem using dynamic programming. This gives a global optimum for the differentiated-pricing problem, assuming the capacity and weights are represented in integer or discretized form.